PHẦN I. JAVA CONCURRENCY

1. Thread Lifecycle và Thread Management

1.1. Khái niệm Thread

Thread là đơn vị thực thi trong một tiến trình. Một chương trình Java có thể sử dụng nhiều thread để xử lý các tác vụ như nhận yêu cầu, đọc dữ liệu, gửi email hoặc chạy công việc nền. Concurrency cho phép nhiều tác vụ cùng tiến triển trong một khoảng thời gian; điều đó không đồng nghĩa mọi tác vụ luôn chạy đồng thời trên các lõi CPU.

Java cung cấp lớp Thread cùng các giao diện Runnable và Callable để biểu diễn công việc. Nên tách phần mô tả tác vụ khỏi cách quản lý thread để chương trình dễ kiểm thử và mở rộng.

```
Runnable task = () -> System.out.println("Task is running");
Thread thread = new Thread(task);
thread.start();
```

Gọi start() yêu cầu JVM khởi chạy một thread mới và thực thi run() trên thread đó. Gọi run() trực tiếp chỉ là lời gọi phương thức thông thường trên thread hiện tại.

1.2. Các trạng thái của Thread

- NEW: thread đã được tạo nhưng chưa gọi start().
- RUNNABLE: thread có thể được JVM lập lịch chạy hoặc đang chạy trên CPU. Java không tách riêng trạng thái RUNNING trong Thread.State.
- BLOCKED: thread đang chờ lấy intrinsic monitor lock để vào một khối hoặc phương thức synchronized.
- WAITING: thread chờ một thread khác thực hiện hành động, chẳng hạn gọi notify(), notifyAll() hoặc hoàn thành khi join() không có thời hạn.
- TIMED_WAITING: thread chờ trong một khoảng thời gian, ví dụ Thread.sleep(), wait(timeout) hoặc join(timeout).
- TERMINATED: phương thức run() đã kết thúc hoặc kết thúc do lỗi không được xử lý.

Trạng thái BLOCKED khác WAITING ở nguyên nhân chờ. BLOCKED thường liên quan đến việc lấy monitor lock; WAITING là chờ một hành động cụ thể từ thread khác. Thread.sleep() không giải phóng monitor lock đang nắm giữ, trong khi Object.wait() giải phóng monitor tương ứng và phải được gọi khi đang giữ monitor đó.

1.3. Quản lý Thread

Một số phương thức thường dùng trong quản lý thread:

- start(): khởi chạy thread.
- sleep(milliseconds): tạm dừng thread hiện tại trong một khoảng thời gian; không giải phóng các monitor lock đang giữ.
- join(): chờ thread được gọi hoàn thành.

- interrupt(): gửi tín hiệu ngắt. Đây không phải lệnh buộc thread dừng ngay; tác vụ cần kiểm tra hoặc xử lý trạng thái ngắt.
- isAlive(): kiểm tra thread đã bắt đầu và chưa kết thúc hay chưa.
- setDaemon(true): đánh dấu thread là daemon trước khi thread được khởi chạy. JVM có thể kết thúc khi chỉ còn daemon thread.

```
Thread worker = new Thread(() -> {
    try {
        while (!Thread.currentThread().isInterrupted()) {
            // Thực hiện một phần công việc
            Thread.sleep(200);
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
    }
});
worker.start();

// Khi cần yêu cầu worker dừng:
worker.interrupt();
worker.join();
```

Trong ứng dụng thực tế, nên ưu tiên ExecutorService thay vì tự tạo quá nhiều thread. Khi cần dừng tác vụ, cần thiết kế cơ chế hủy rõ ràng, xử lý InterruptedException đúng cách và tránh bỏ qua tín hiệu interrupt.

2. ThreadPool Patterns và ExecutorService

2.1. Vì sao cần Thread Pool?

Tạo một thread mới cho mỗi tác vụ gây chi phí khởi tạo và quản lý, đồng thời có thể tạo ra quá nhiều thread khi tải tăng. Thread pool tái sử dụng một nhóm thread để thực thi nhiều tác vụ, giúp kiểm soát tài nguyên và tách việc gửi tác vụ khỏi việc thực thi.

2.2. ExecutorService

ExecutorService là API quản lý việc thực thi tác vụ bất đồng bộ. submit() có thể trả về Future để lấy kết quả hoặc kiểm tra trạng thái; execute() gửi tác vụ Runnable mà không trả về Future.

```
ExecutorService executor = Executors.newFixedThreadPool(4);
try {
    Future<Integer> result = executor.submit(() -> 10 + 20);
    System.out.println(result.get());
} finally {
    executor.shutdown();
}
```

shutdown() ngừng nhận tác vụ mới nhưng vẫn cho các tác vụ đã gửi tiếp tục hoàn thành. shutdownNow() cố gắng dừng các tác vụ đang chạy bằng cơ chế interrupt và trả về các tác vụ chưa bắt đầu; nó không bảo đảm mọi tác vụ sẽ dừng tức thì. Có thể dùng awaitTermination() để chờ pool kết thúc trong thời hạn nhất định.

2.3. Các loại Thread Pool thường gặp

- Fixed Thread Pool: số lượng thread cố định, phù hợp khi muốn giới hạn số tác vụ chạy đồng thời. Với một số factory phổ biến, hàng đợi có thể không giới hạn nên vẫn cần cân nhắc nguy cơ tích tụ tác vụ.
- Cached Thread Pool: tạo thêm thread khi cần và tái sử dụng thread rảnh; không phù hợp với mọi tải vì số thread có thể tăng đáng kể.
- Single Thread Executor: thực thi lần lượt các tác vụ trên một thread, hữu ích khi cần thứ tự xử lý.
- Scheduled Thread Pool: lập lịch chạy tác vụ sau một khoảng thời gian hoặc lặp định kỳ.
- ThreadPoolExecutor tùy chỉnh: cho phép thiết lập corePoolSize, maximumPoolSize, keepAliveTime, hàng đợi và RejectedExecutionHandler.

2.4. Các mô hình sử dụng phổ biến

Worker Pool dùng một nhóm worker để xử lý hàng đợi công việc. Producer–Consumer tách bên tạo dữ liệu khỏi bên xử lý dữ liệu, thường dùng BlockingQueue để trao đổi an toàn giữa các thread. Bulkhead cô lập các nhóm tác vụ bằng pool hoặc semaphore riêng để sự cố hay quá tải ở một nhóm không làm cạn toàn bộ tài nguyên của ứng dụng.

CallerRunsPolicy là một chính sách từ chối của ThreadPoolExecutor: khi pool không nhận thêm được tác vụ và executor chưa bị shutdown, thread gọi submit/execute có thể trực tiếp chạy tác vụ đó. Điều này tạo áp lực ngược nhưng cũng có thể khiến thread chính bị chậm.

3. Synchronization: synchronized, Lock và Atomic Classes

3.1. Race Condition và Thread Safety

Race condition xảy ra khi kết quả phụ thuộc vào thứ tự thực thi không được kiểm soát giữa các thread. Ví dụ, phép tăng count++ gồm đọc giá trị, cộng một và ghi lại; nhiều thread có thể đọc cùng giá trị cũ rồi ghi đè kết quả của nhau.

```
class Counter {
    private int count = 0;

    public synchronized void increment() {
        count++;
    }

    public synchronized int getCount() {
        return count;
    }
}
```

Từ khóa synchronized đảm bảo tại một thời điểm chỉ một thread nắm giữ cùng monitor mới có thể vào vùng synchronized tương ứng. Nó cũng thiết lập quan hệ visibility giữa việc nhả và lấy cùng monitor, giúp các thay đổi được công bố đúng cách.

3.2. synchronized

synchronized có thể áp dụng cho phương thức hoặc khối lệnh. Phương thức synchronized thông thường khóa monitor của đối tượng this; phương thức static synchronized khóa monitor của đối tượng Class tương ứng. Khối synchronized cho phép chọn đối tượng khóa cụ thể.

```
private final Object lock = new Object();
private int value;

public void update(int newValue) {
    synchronized (lock) {
        value = newValue;
    }
}
```

Cần tránh giữ lock trong thời gian dài hoặc gọi các thao tác chậm khi đang giữ lock. Thiết kế khóa không hợp lý có thể gây contention hoặc deadlock.

3.3. Lock và ReentrantLock

Giao diện Lock cung cấp khả năng khóa linh hoạt hơn synchronized, bao gồm tryLock(), lockInterruptibly() và tùy chọn công bằng ở một số implementation. Khi dùng Lock, phải bảo đảm unlock() trong finally để tránh giữ khóa ngoài ý muốn.

```
private final Lock lock = new ReentrantLock();
private int count;

public void increment() {
    lock.lock();
    try {
        count++;
    } finally {
        lock.unlock();
    }
}
```

tryLock() cho phép thử lấy khóa mà không chờ vô hạn. ReentrantLock cho phép cùng thread lấy lại khóa mà nó đang sở hữu, nhưng vẫn cần cân nhắc thứ tự lấy nhiều khóa để giảm nguy cơ deadlock.

3.4. Atomic Classes

Các lớp trong java.util.concurrent.atomic như AtomicInteger, AtomicLong và AtomicReference cung cấp thao tác nguyên tử trên một biến. Chúng phù hợp với các cập nhật độc lập, chẳng hạn bộ đếm, mà không cần khóa tường minh trong nhiều trường hợp.

```
AtomicInteger counter = new AtomicInteger(0);
counter.incrementAndGet();
int current = counter.get();
```

AtomicInteger bảo đảm phép incrementAndGet() là nguyên tử, nhưng nhiều thao tác phối hợp với nhau vẫn có thể cần khóa hoặc một thiết kế đồng bộ khác. Atomic không tự động làm cho toàn bộ đối tượng hay quy trình nghiệp vụ trở nên thread-safe.

3.5. So sánh và lựa chọn

Dùng synchronized cho vùng tới hạn đơn giản, dễ đọc và dễ bảo trì. Dùng Lock khi cần thử khóa, khả năng ngắt trong lúc chờ hoặc các tính năng điều khiển bổ sung. Dùng Atomic classes cho cập nhật nguyên tử trên một biến hoặc tham chiếu. Lựa chọn phải dựa trên tính đúng đắn và đặc điểm tải, không chỉ dựa trên kỳ vọng hiệu năng.

4. CompletableFuture và Reactive Programming Basics

4.1. CompletableFuture

CompletableFuture biểu diễn kết quả của một tác vụ có thể hoàn thành trong tương lai và cho phép nối các bước xử lý mà không cần chặn thread ở mỗi bước. Các phương thức thenApply(), thenAccept(), thenRun(), thenCompose() và thenCombine() hỗ trợ những cách kết hợp khác nhau.

- thenApply(): nhận kết quả trước đó, biến đổi và trả về kết quả mới.
- thenAccept(): nhận kết quả để thực hiện hành động, không trả về giá trị mới.
- thenRun(): chạy hành động không nhận kết quả trước đó.
- thenCompose(): nối một bước trả về CompletableFuture khác, tránh tạo cấu trúc lồng nhau.
- thenCombine(): kết hợp kết quả từ hai future độc lập.
- allOf(): hoàn thành khi tất cả future được cung cấp hoàn thành, trả về CompletableFuture<Void>.
- anyOf(): hoàn thành khi một future bất kỳ hoàn thành đầu tiên, kết quả có kiểu Object.

```
CompletableFuture<Integer> price =
    CompletableFuture.supplyAsync(() -> 100);

CompletableFuture<String> message =
    price.thenApply(p -> "Price: " + p);

message.thenAccept(System.out::println).join();
```

Các phương thức Async thường thực hiện callback trên executor được chỉ định hoặc trên executor mặc định nếu không chỉ định. Khi dùng trong ứng dụng backend, nên cân nhắc cung cấp executor phù hợp, đặc biệt với tác vụ chặn I/O, để tránh làm cạn pool dùng chung.

Cần xử lý lỗi bằng exceptionally(), handle() hoặc whenComplete() tùy mục đích. join() chờ kết quả và thường ném CompletionException nếu tác vụ hoàn thành lỗi; get() có thể ném các ngoại lệ đã kiểm tra như ExecutionException và InterruptedException.

4.2. Reactive Programming

Reactive Programming mô tả dữ liệu và sự kiện như một luồng bất đồng bộ, cho phép các bước xử lý được kết nối với nhau. Trong Java, Reactor và RxJava là những thư viện thường gặp; Spring WebFlux sử dụng mô hình reactive với các kiểu như Mono và Flux.

- Mono<T>: biểu diễn tối đa một phần tử hoặc một kết quả rỗng.
- Flux<T>: biểu diễn từ không đến nhiều phần tử.
- Backpressure: cơ chế để bên tiêu thụ báo khả năng tiếp nhận dữ liệu, giúp kiểm soát luồng dữ liệu khi tốc độ sản xuất lớn hơn tốc độ xử lý.
- Non-blocking I/O: cho phép một thread xử lý công việc khác trong lúc chờ I/O, nếu toàn bộ chuỗi xử lý được thiết kế không chặn.

Reactive không đồng nghĩa với việc mọi đoạn code tự động nhanh hơn. Nếu gọi thao tác blocking trên thread xử lý event loop, hiệu quả có thể giảm đáng kể. Mô hình này phù hợp với ứng dụng có nhiều I/O đồng thời, nhưng đòi hỏi hiểu rõ cách xử lý lỗi, luồng dữ liệu và backpressure.

PHẦN II. DOCKER

5. Container Concepts và Docker Architecture

5.1. Khái niệm Docker và Container

Docker là nền tảng dùng để đóng gói, phân phối và chạy ứng dụng trong container. Container chứa ứng dụng cùng các thư viện và phụ thuộc cần thiết, giúp môi trường chạy nhất quán hơn giữa máy phát triển, môi trường kiểm thử và máy chủ.

Container thường chia sẻ kernel của hệ điều hành host nhưng được cô lập bằng các cơ chế của hệ điều hành như namespaces và cgroups trên Linux. Container không phải là một máy ảo đầy đủ và không mặc định có kernel riêng.

5.2. Các thành phần chính của Docker

- Docker Client: giao diện dòng lệnh hoặc công cụ gửi yêu cầu Docker, ví dụ lệnh docker build hay docker run.
- Docker Daemon (dockerd): dịch vụ nền quản lý image, container, network và volume.
- Docker Engine API: giao diện để client giao tiếp với daemon.
- Docker Images: mẫu chỉ đọc dùng để tạo container.
- Docker Containers: các instance được tạo từ image, có lớp ghi riêng khi chạy.
- Docker Registry: nơi lưu trữ và phân phối image, ví dụ Docker Hub hoặc registry riêng.

Khi thực hiện docker run, Docker tìm hoặc tải image nếu cần, tạo container với cấu hình được yêu cầu rồi khởi chạy tiến trình chính bên trong container.

6. Container và Virtual Machine

Container và máy ảo đều hỗ trợ cô lập ứng dụng, nhưng khác nhau ở lớp ảo hóa và mức tài nguyên sử dụng.

- Container thường dùng chung kernel của host, vì vậy thường khởi động nhanh và có overhead thấp hơn.
- Virtual Machine (VM) chạy hệ điều hành khách với kernel riêng trên hypervisor, cung cấp ranh giới hệ điều hành độc lập hơn nhưng thường cần nhiều tài nguyên hơn.
- Container cô lập tiến trình và tài nguyên nhưng mức cô lập phụ thuộc cấu hình host, runtime, quyền và chính sách bảo mật.
- VM hữu ích khi cần chạy các hệ điều hành khác nhau hoặc cần ranh giới kernel riêng; container phù hợp để đóng gói và triển khai ứng dụng nhanh, nhất quán.

Container không thay thế hoàn toàn VM. Trong môi trường thực tế, container thường chạy trên các máy ảo hoặc máy chủ được quản lý, kết hợp ưu điểm của hai mô hình.

7. Dockerfile Best Practices

7.1. Vai trò của Dockerfile

Dockerfile là tệp mô tả các bước xây dựng Docker image. Các chỉ thị thường gặp gồm FROM, WORKDIR, COPY, RUN, EXPOSE, ENV, USER và ENTRYPOINT.

```
FROM eclipse-temurin:21-jre
WORKDIR /app
COPY target/app.jar app.jar
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Ví dụ trên dùng image Java Runtime để chạy ứng dụng JAR. WORKDIR đặt thư mục làm việc; COPY sao chép JAR vào image; EXPOSE ghi nhận cổng ứng dụng dự kiến lắng nghe trong container; ENTRYPOINT chỉ định tiến trình chính. EXPOSE không tự động công khai cổng ra host; muốn truy cập từ host thường cần cấu hình -p khi chạy container.

7.2. Các thực hành tốt

- Chọn image nền phù hợp và ưu tiên phiên bản cụ thể, được cập nhật bảo mật; cân nhắc image tối giản khi tương thích.
- Dùng multi-stage build khi cần biên dịch ứng dụng để không đưa toàn bộ công cụ build vào image runtime.
- Sắp xếp các chỉ thị ít thay đổi trước các chỉ thị thường thay đổi để tận dụng cache.
- Tạo tệp .dockerignore để loại bỏ các tệp không cần thiết như .git, thư mục build thừa hoặc dữ liệu cục bộ.
- Không đưa mật khẩu, token hoặc khóa bí mật vào Dockerfile hay image; sử dụng cơ chế secrets phù hợp với môi trường triển khai.
- Chạy ứng dụng bằng user không phải root nếu có thể.
- Giữ container tập trung vào một dịch vụ hoặc trách nhiệm chính và để tiến trình chính chạy ở foreground.
- Sử dụng dạng exec cho ENTRYPOINT hoặc CMD khi chạy ứng dụng để tín hiệu hệ điều hành được chuyển tiếp phù hợp.

7.3. Ví dụ multi-stage build

```
FROM maven:3.9-eclipse-temurin-21 AS build
WORKDIR /src
COPY pom.xml .
COPY src ./src
RUN mvn -q -DskipTests package

FROM eclipse-temurin:21-jre
WORKDIR /app
COPY --from=build /src/target/*.jar app.jar
USER 10001
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

Giai đoạn build sử dụng Maven để tạo JAR. Giai đoạn runtime chỉ sao chép artifact cần thiết sang image Java Runtime, nhờ đó giảm các công cụ không cần thiết trong image cuối cùng. Trong dự án thực tế, cần bảo đảm tên artifact và cấu hình build khớp với dự án.

8. Image Layers và Optimization

8.1. Image Layer và Union Filesystem

Docker image được xây dựng từ nhiều layer. Nhiều chỉ thị build có thể tạo layer hoặc metadata tương ứng; các layer của image được quản lý theo cơ chế lưu trữ phân lớp. Union filesystem hoặc cơ chế snapshot tương đương cho phép các lớp được kết hợp thành một hệ thống tệp nhìn thấy được khi container chạy.

Image layer thường bất biến và có thể được chia sẻ giữa nhiều image nếu nội dung và định danh lớp trùng nhau. Khi container chạy, Docker cung cấp thêm lớp ghi riêng cho container. Thay đổi trong lớp ghi này không tự động cập nhật image gốc.

8.2. Layer Caching

Docker build có thể tái sử dụng cache của các bước trước nếu chỉ thị và dữ liệu đầu vào liên quan không thay đổi. Khi một bước bị thay đổi, những bước phía sau có thể phải chạy lại. Vì vậy, nên sao chép các tệp quản lý dependency trước, cài dependency, rồi mới sao chép phần mã nguồn thường xuyên thay đổi.

```
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src ./src
RUN mvn package
```

Cách tối ưu cụ thể phụ thuộc công cụ build và cấu hình dự án. Mục tiêu là tránh tải lại dependency không cần thiết, giảm kích thước image cuối cùng và giảm số lớp dữ liệu thừa.

8.3. Các cách tối ưu image

- Dùng image nền nhỏ nhưng vẫn đáp ứng yêu cầu tương thích và bảo mật.
- Sử dụng multi-stage build để tách môi trường build khỏi môi trường runtime.
- Loại bỏ tệp tạm, cache package và công cụ không cần trong image cuối.
- Tận dụng cache hợp lý và hạn chế thay đổi không cần thiết ở các bước đầu.
- Dùng .dockerignore để tránh gửi dữ liệu không liên quan vào build context.
- Kiểm tra kích thước image, lỗ hổng và các thành phần không cần thiết trước khi triển khai.

9. Container Lifecycle

Vòng đời container bao gồm các thao tác tạo, khởi chạy, dừng, khởi động lại và xóa. Image là mẫu để tạo container; container là instance có trạng thái và lớp ghi riêng.

- create: tạo container từ image nhưng chưa khởi chạy tiến trình chính.
- start: khởi chạy container đã được tạo.
- run: thao tác kết hợp tạo và khởi chạy container; có thể thêm tùy chọn cấu hình cổng, biến môi trường và volume.
- stop: gửi tín hiệu yêu cầu tiến trình dừng và chờ trong khoảng thời gian cho phép; sau đó có thể bị buộc kết thúc.
- restart: khởi động lại container.

- rm: xóa container đã dừng. Cần cân nhắc dữ liệu nằm trong lớp ghi của container vì dữ liệu đó có thể mất khi container bị xóa.
- images: liệt kê image hiện có; rmi dùng để xóa image khi không còn bị tham chiếu.

```
docker build -t my-java-app:1.0 .
docker create --name java-app my-java-app:1.0
docker start java-app
docker logs java-app
docker stop java-app
docker rm java-app
```

Nên kiểm tra log và trạng thái container khi ứng dụng không khởi động. Có thể sử dụng docker ps -a, docker logs và docker inspect để tìm nguyên nhân.

10. Container Networking

10.1. Mục đích của Docker Networking

Docker networking cho phép container giao tiếp với container khác, với host và với mạng bên ngoài. Mạng được cấu hình phù hợp giúp tách biệt dịch vụ, giới hạn phạm vi truy cập và quản lý cách các ứng dụng tìm thấy nhau.

10.2. Các loại network phổ biến

- Bridge: kiểu mạng phổ biến trên một Docker host. Các container trên cùng user-defined bridge có thể giao tiếp với nhau; tên container hoặc alias có thể được phân giải thông qua DNS tích hợp của Docker.
- Host: container dùng network namespace của host trên nền tảng hỗ trợ, giảm lớp NAT nhưng giảm mức cô lập mạng. Hành vi cụ thể khác nhau theo hệ điều hành và môi trường chạy Docker.
- Overlay: kết nối các container trên nhiều Docker host, thường dùng trong môi trường điều phối như Docker Swarm.
- None: không cấu hình kết nối mạng thông thường cho container, phù hợp với một số tác vụ đặc biệt.

10.3. Publish port và giao tiếp dịch vụ

Lệnh -p ánh xạ một cổng của host tới cổng bên trong container. Ví dụ -p 8080:8080 cho phép truy cập dịch vụ container qua cổng 8080 của host nếu ứng dụng lắng nghe đúng địa chỉ và không có quy tắc mạng chặn truy cập.

```
docker run -d --name api -p 8080:8080 my-java-app:1.0
```

Trong Docker Compose, các service cùng network thường gọi nhau bằng tên service và cổng container. Ví dụ backend có thể kết nối database bằng hostname db:5432 trong mạng nội bộ, thay vì dùng localhost. Bên trong container backend, localhost trỏ về chính container backend.

11. Storage: Bind Mounts, Volumes và tmpfs

Lớp ghi của container phù hợp với dữ liệu tạm thời gắn với vòng đời container. Dữ liệu cần tồn tại sau khi container bị xóa nên được đặt trong volume hoặc bind mount tùy trường hợp.

11.1. Bind Mount

Bind mount ánh xạ một đường dẫn cụ thể trên host vào đường dẫn bên trong container. Nó hữu ích trong phát triển cục bộ, ví dụ mount mã nguồn để container đọc trực tiếp. Đường dẫn phụ thuộc hệ điều hành host và cần chú ý quyền truy cập.

```
docker run --rm -v "$PWD:/app" -w /app eclipse-temurin:21-jdk java -version
```

Ví dụ cú pháp đường dẫn có thể khác trên Windows PowerShell, Linux hoặc macOS; cần dùng đúng cú pháp của môi trường đang chạy.

11.2. Docker Volume

Volume được Docker quản lý và phù hợp để lưu dữ liệu lâu dài, ví dụ dữ liệu của database. Volume có thể được gắn vào container khác để dữ liệu tiếp tục tồn tại khi container ban đầu bị xóa.

```
docker volume create db-data
docker run -d --name database \
  -v db-data:/var/lib/postgresql/data \
  postgres:16
```

Cần sao lưu và quản lý vòng đời volume riêng. Xóa container không đồng nghĩa volume bị xóa; lệnh xóa volume cần được thực hiện có chủ đích.

11.3. tmpfs

tmpfs mount lưu dữ liệu tạm thời trong bộ nhớ trên các môi trường hỗ trợ. Dữ liệu không được giữ lại sau khi container dừng hoặc khởi động lại theo vòng đời tmpfs. Nó phù hợp với dữ liệu tạm hoặc nhạy cảm cần tránh ghi xuống lớp lưu trữ bền vững, nhưng vẫn cần đánh giá giới hạn và cấu hình bảo mật.

```
docker run --rm --tmpfs /tmp my-java-app:1.0
```

11.4. So sánh

- Bind mount: đường dẫn do người dùng chọn trên host; thuận tiện khi phát triển và chia sẻ tệp cụ thể.
- Volume: Docker quản lý vị trí lưu trữ; phù hợp với dữ liệu ứng dụng cần tồn tại độc lập với container.
- tmpfs: dữ liệu tạm trong bộ nhớ; không phù hợp làm nơi lưu dữ liệu cần giữ sau khi container kết thúc.

12. Docker Compose cho ứng dụng nhiều container

12.1. Khái niệm

Docker Compose cho phép định nghĩa và quản lý nhiều service trong cùng một tệp cấu hình YAML. Một hệ thống backend phổ biến có thể gồm ứng dụng Java, database và công cụ hỗ trợ. Compose giúp cấu hình các service, network, volume và biến môi trường tập trung, thuận tiện cho môi trường phát triển và kiểm thử.

12.2. Ví dụ cấu hình

```
services:
  backend:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: appdb
      DB_USER: appuser
      DB_PASSWORD: ${DB_PASSWORD}
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: appdb
      POSTGRES_USER: appuser
      POSTGRES_PASSWORD: ${DB_PASSWORD}
    volumes:
      - postgres-data:/var/lib/postgresql/data

volumes:
  postgres-data:
```

Trong ví dụ, backend được build từ Dockerfile hiện tại và công khai cổng 8080. Service db sử dụng PostgreSQL và volume postgres-data để lưu dữ liệu. Backend truy cập database qua hostname db vì Compose cung cấp DNS nội bộ cho các service trên cùng network.

depends_on xác định thứ tự khởi tạo service nhưng ở dạng cơ bản không bảo đảm database đã sẵn sàng nhận kết nối. Ứng dụng nên có cơ chế retry kết nối hoặc cấu hình healthcheck và điều kiện phụ thuộc phù hợp khi phiên bản Compose hỗ trợ.

12.3. Các lệnh Compose thường dùng

- docker compose up --build: build image cần thiết và khởi chạy các service.
- docker compose up -d: chạy các service ở chế độ nền.
- docker compose ps: xem trạng thái service và container.
- docker compose logs -f: theo dõi log của các service.
- docker compose exec backend sh: mở shell trong container backend nếu image có shell.
- docker compose down: dừng và xóa các container cùng network do Compose tạo.
- docker compose down -v: ngoài container và network, xóa cả các volume được quản lý trong phạm vi lệnh; cần dùng thận trọng vì có thể làm mất dữ liệu.

TÀI LIỆU THAM KHẢO

- Oracle Java Documentation: https://docs.oracle.com/en/java/

- Java Concurrency Utilities: https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/package-summary.html
- Docker Documentation: https://docs.docker.com/
- Dockerfile reference: https://docs.docker.com/reference/dockerfile/
- Docker Compose documentation: https://docs.docker.com/compose/